

**BỘ GIÁO DỤC VÀ ĐÀO TẠO  
TRƯỜNG ĐẠI HỌC PHENIKAA**

---



**BÁO CÁO NGHIÊN CỨU KHOA HỌC**

**HỆ THỐNG GỢI Ý SẢN PHẨM HAI GIAI ĐOẠN  
TÍCH HỢP TRIỆU HỒI DUAL-CHANNEL VÀ ĐẶC  
TRƯNG PHIÊN POINT-IN-TIME**

|                              |                                   |
|------------------------------|-----------------------------------|
| <b>Chủ nhiệm đề tài:</b>     | Nguyễn Tuấn Anh                   |
| <b>Mã số sinh viên:</b>      | 24101779                          |
| <b>Ngành:</b>                | Tài năng Khoa học máy tính        |
| <b>Thành viên thực hiện:</b> | Phạm Đình Trường<br>Đoàn Văn Hiệp |
| <b>Giảng viên hướng dẫn:</b> | TS. Dương Thị Kim Huyền           |

**HÀ NỘI, 05/2026**

## **Lời cảm ơn**

Trước khi trình bày nội dung chính của báo cáo này, chúng em xin bày tỏ lòng biết ơn sâu sắc tới TS. Dương Thị Kim Huyền - Trường Đại học Phenikaa đã trực tiếp hướng dẫn, định hướng khoa học và tận tình chỉ bảo trong suốt quá trình nghiên cứu để chúng em có thể phát triển hoàn thiện hệ thống này.

Chúng em cũng xin bày tỏ lòng biết ơn chân thành tới toàn thể các thầy cô giáo tại Trường Đại học Phenikaa, đặc biệt là các giảng viên ngành Tài năng Khoa học máy tính đã truyền đạt tri thức quý báu, định hướng nghiên cứu và tạo mọi điều kiện thuận lợi để chúng em thực hiện đồ án.

Cuối cùng, chúng em xin được gửi lời cảm ơn chân thành tới gia đình, bạn bè và các đồng nghiệp đã luôn ủng hộ, chia sẻ và đồng hành cùng chúng em trong suốt thời gian nghiên cứu và hoàn thành báo cáo này.

Chúng em xin chân thành cảm ơn!

Hà Nội, tháng 05 năm 2026

**Tập thể tác giả thực hiện**

## **Lời cam đoan**

Chúng em xin cam đoan đây là công trình nghiên cứu của riêng nhóm chúng em dưới sự hướng dẫn khoa học của TS. Dương Thị Kim Huyền. Tất cả các kết quả đo lường, kết quả đánh giá mô hình và mã nguồn hệ thống được trình bày trong báo cáo này đều mang tính trung thực, được thử nghiệm khách quan và chưa từng được công bố ở bất kỳ công trình nghiên cứu nào khác. Các tài liệu tham khảo có liên quan đều được trích dẫn và liệt kê đầy đủ trong danh mục tài liệu tham khảo.

*Hà Nội, ngày... tháng... năm 2026*

**GIẢNG VIÊN HƯỚNG DẪN**

*(Ký, ghi rõ họ tên)*

**ĐẠI DIỆN TÁC GIẢ**

*(Ký, ghi rõ họ tên)*

## **Tóm tắt nghiên cứu**

Đồ án này tập trung nghiên cứu, thiết kế và phát triển hệ thống gợi ý sản phẩm thương mại điện tử hai giai đoạn (Two-Stage Recommender System) quy mô lớn đạt độ trễ cực thấp ( $< 50\text{ms}$ ) và độ chính xác cá nhân hóa cao. Hệ thống kết hợp cơ chế Triệu hồi song song hai kênh (Dual-Channel Recall) trên nền tảng học sâu Matrix Factorization của PyTorch cùng tìm kiếm vector lân cận gần nhất siêu tốc bằng thư viện FAISS, với giai đoạn xếp hạng chi tiết bằng thuật toán học máy dạng cây LightGBM. Để loại bỏ hiện tượng rò rỉ dữ liệu tương lai, chúng em áp dụng nguyên lý trích xuất đặc trưng Point-in-time lũy tiến. Ngoài ra, lỗi lệch phân phối dữ liệu (Train-Test Distribution Mismatch) của giai đoạn xếp hạng được giải quyết triệt để thông qua kịch bản huấn luyện trên ứng viên thực tế (Train-on-Recall). Kết quả thử nghiệm cho thấy mô hình cải tiến giúp tăng vọt chỉ số Validation AUC lên 0.9601 (+50.6%) và NDCG@10 lên 0.0653 (+31.9%) trong môi trường dữ liệu cực kỳ thưa thớt thực tế.

**Từ khóa:** Hệ thống gợi ý hai giai đoạn, Triệu hồi hai kênh, Matrix Factorization, FAISS, LightGBM, Point-in-time, Train-on-Recall.

# Mục lục

|                                                                              |            |
|------------------------------------------------------------------------------|------------|
| <b>Lời cảm ơn</b>                                                            | <b>i</b>   |
| <b>Lời cam đoan</b>                                                          | <b>ii</b>  |
| <b>Tóm tắt nghiên cứu</b>                                                    | <b>iii</b> |
| <b>1 Phần Mở đầu</b>                                                         | <b>1</b>   |
| 1.1 Bối cảnh bài toán và lý do thực hiện dự án . . . . .                     | 1          |
| 1.2 Mục tiêu của hệ thống gợi ý . . . . .                                    | 1          |
| 1.3 Phạm vi áp dụng và giới hạn của dự án . . . . .                          | 2          |
| <b>2 Cơ sở Lý thuyết</b>                                                     | <b>3</b>   |
| 2.1 Tổng quan về Hệ thống Gợi ý (Recommender Systems) . . . . .              | 3          |
| 2.2 Kiến trúc Hệ thống Gợi ý 2 giai đoạn (Two-Stage Architecture) . . . . .  | 3          |
| 2.3 Các phương pháp Candidate Generation (Recall Model) . . . . .            | 4          |
| 2.4 Các phương pháp Ranking (Ranking Model) và thuật toán LightGBM . . . . . | 4          |
| 2.5 Các chỉ số đánh giá hiệu suất (Evaluation Metrics) . . . . .             | 5          |
| <b>3 Quy trình Xử lý và Phân tích Dữ liệu</b>                                | <b>6</b>   |
| 3.1 Phân tích Dữ liệu Khám phá (Exploratory Data Analysis - EDA) . . . . .   | 6          |
| 3.2 Xử lý luồng dữ liệu (Data Pipeline) và cơ chế Sessionizer . . . . .      | 6          |
| 3.3 Kỹ thuật trích xuất đặc trưng (Feature Engineering) . . . . .            | 7          |
| 3.4 Gán nhãn tương tác ngầm (Pseudo-Labeling) . . . . .                      | 7          |
| <b>4 Thiết kế và Huấn luyện Mô hình</b>                                      | <b>9</b>   |
| 4.1 Kiến trúc tổng thể của hệ thống . . . . .                                | 9          |
| 4.2 Xây dựng và tối ưu mô hình Recall . . . . .                              | 9          |
| 4.3 Xây dựng và huấn luyện mô hình Ranking . . . . .                         | 10         |
| 4.4 Giải pháp Train-on-Recall và Kết quả thử nghiệm . . . . .                | 10         |
| <b>5 Triển khai Hệ thống Phục vụ (Serving Pipeline)</b>                      | <b>12</b>  |

|          |                                                                          |           |
|----------|--------------------------------------------------------------------------|-----------|
| 5.1      | Thiết kế luồng phục vụ hệ thống (System Flow) . . . . .                  | 12        |
| 5.2      | Truy xuất vector lân cận gần nhất siêu tốc bằng FAISS Index . . . . .    | 13        |
| 5.3      | Tích hợp Pipeline và cơ chế Ranker thời gian thực kháng lỗi . . . . .    | 13        |
| 5.4      | Kiểm thử hệ thống (Smoke Testing) . . . . .                              | 14        |
| <b>6</b> | <b>Kết luận và Hướng phát triển</b>                                      | <b>15</b> |
| 6.1      | Tổng kết các kết quả đạt được . . . . .                                  | 15        |
| 6.2      | Phân tích các vấn đề dữ liệu còn tồn đọng (Data Issues) . . . . .        | 15        |
| 6.3      | Kế hoạch cải thiện dữ liệu và hệ thống (Data Improvement Plan) . . . . . | 16        |
| 6.3.1    | Kế hoạch cải thiện Dữ liệu (Data Improvement Plan) . . . . .             | 16        |
| 6.3.2    | Kế hoạch nâng cấp Hệ thống (System Improvement Plan) . . . . .           | 16        |

## **Danh sách hình vẽ**

# Danh sách bảng

|     |                                                                                             |    |
|-----|---------------------------------------------------------------------------------------------|----|
| 4.1 | Bảng so sánh hiệu năng xếp hạng giữa Baseline và mô hình cải tiến Train-on-Recall . . . . . | 11 |
| 5.1 | Bảng đo lường độ trễ chi tiết các bước trong API Serving trực tuyến . . . .                 | 14 |



# Chương 1

## Phần Mở đầu

### 1.1 Bối cảnh bài toán và lý do thực hiện dự án

Trong thời đại số hóa, thương mại điện tử (e-commerce) đã trở thành một phần không thể thiếu trong đời sống tiêu dùng toàn cầu. Với hàng triệu sản phẩm đa dạng trên các kệ hàng trực tuyến, khách hàng thường xuyên rơi vào trạng thái quá tải thông tin (information overload) khi tìm kiếm sản phẩm phù hợp. Ngược lại, doanh nghiệp cũng gặp khó khăn trong việc tiếp cận và giới thiệu đúng sản phẩm đến đối tượng khách hàng tiềm năng.

Hệ thống gợi ý (Recommender Systems) ra đời như một giải pháp đột phá giải quyết mâu thuẫn này. Tuy nhiên, việc xây dựng một hệ thống gợi ý quy mô lớn trong môi trường công nghiệp thực tế đòi hỏi phải xử lý đồng thời hai thách thức:

1. **Độ trễ phục vụ cực thấp ( $< 50ms$ ):** Đảm bảo trải nghiệm lướt ứng dụng mượt mà, không gián đoạn cho người dùng dưới điều kiện tải cao.
2. **Độ chính xác cá nhân hóa cao:** Khai thác hiệu quả hành vi tương tác lịch sử và bối cảnh tức thì của phiên mua sắm.

Dự án nghiên cứu này được thực hiện nhằm xây dựng một hệ thống gợi ý hai giai đoạn kết hợp triệu hồi song song và xếp hạng chi tiết để giải quyết trọn vẹn hai yêu cầu khắt khe trên.

### 1.2 Mục tiêu của hệ thống gợi ý

Mục tiêu cụ thể của dự án bao gồm:

- **Về mặt kiến trúc:** Thiết kế hệ thống hai giai đoạn (Two-Stage Architecture) tiêu chuẩn công nghiệp nhằm chia tách hiệu quả bài toán lọc thô (Recall) và xếp hạng chi tiết (Ranking).

- **Về mặt thuật toán:**

- Huấn luyện mô hình học sâu Matrix Factorization trên PyTorch kết hợp chỉ mục FAISS tìm kiếm KNN siêu tốc để thu hẹp không gian ứng viên thô.
- Huấn luyện mô hình cây quyết định LightGBM chấm điểm xác suất chuyển đổi chính xác.

- **Về mặt kỹ thuật dữ liệu:** Triển khai cơ chế trích xuất đặc trưng lũy kế Point-in-time triệt tiêu hoàn toàn rò rỉ dữ liệu tương lai và áp dụng giải pháp Train-on-Recall xóa bỏ lỗi lệch phân phối dữ liệu (Train-Test Distribution Shift).

- **Về mặt triển khai serving:** Xây dựng API FastAPI hiệu năng cao, tải sẵn dữ liệu lên RAM, phản hồi trực tuyến với độ trễ thấp ( $< 25\text{ms}$ ) và có tính kháng lỗi cao.

### 1.3 Phạm vi áp dụng và giới hạn của dự án

- **Phạm vi áp dụng:** Hệ thống được thiết kế tối ưu cho các nền tảng thương mại điện tử quy mô vừa đến lớn, nơi có lượng sản phẩm từ hàng chục nghìn đến hàng triệu và hành vi người dùng đa dạng (Xem, Thêm giỏ, Mua hàng).

- **Giới hạn của dự án:**

- Tập trung tối ưu hóa các tương tác dạng bảng và lịch sử hành vi (implicit feedback), chưa khai thác chuyên sâu dữ liệu phi cấu trúc như hình ảnh sản phẩm hoặc ngôn ngữ tự nhiên mô tả chi tiết sản phẩm.
- Đánh giá offline được thực hiện trên tập snapshot dữ liệu tương tác thực tế, các đánh giá online (A/B Testing) cần được triển khai kiểm chứng thực tế khi tích hợp vào môi trường doanh nghiệp.

# Chương 2

## Cơ sở Lý thuyết

### 2.1 Tổng quan về Hệ thống Gợi ý (Recommender Systems)

Hệ thống gợi ý (Recommender Systems) là một nhánh quan trọng của Trí tuệ Nhân tạo và Học máy, được thiết kế nhằm dự đoán mức độ quan tâm của người dùng đối với một sản phẩm hoặc nội dung cụ thể. Các phương pháp gợi ý truyền thống phổ biến bao gồm:

- **Lọc dựa trên nội dung (Content-based Filtering):** Gợi ý sản phẩm tương đồng về đặc tính (thương hiệu, chất liệu, danh mục) với các mặt hàng người dùng đã thích trong quá khứ. Hạn chế lớn là khó mở rộng và không gợi ý được các mặt hàng nằm ngoài gu cũ (vấn đề quá chuyên biệt).
- **Lọc cộng tác (Collaborative Filtering):** Dựa trên giả định rằng những người dùng có chung hành vi trong quá khứ sẽ có xu hướng lựa chọn giống nhau trong tương lai. Lọc cộng tác chia làm hai dạng: User-based (so khớp độ tương đồng giữa các user) và Item-based (so khớp tương đồng giữa các sản phẩm dựa trên lượt đánh giá của chung người dùng).

### 2.2 Kiến trúc Hệ thống Gợi ý 2 giai đoạn (Two-Stage Architecture)

Khi số lượng sản phẩm tăng lên hàng triệu, việc tính toán chi tiết độ tương thích giữa một người dùng với từng sản phẩm bằng các mô hình học máy phức tạp trở nên bất khả thi do giới hạn độ trễ phục vụ trực tuyến. Để giải quyết bài toán quy mô lớn này, kiến trúc gợi ý Hai Giai Đoạn được đề xuất và áp dụng rộng rãi:

1. **Giai đoạn Triệu hồi (Candidate Generation / Recall):** Thực hiện sàng lọc thô siêu

tốc. Từ hàng triệu sản phẩm, sử dụng các thuật toán đơn giản hoặc tìm kiếm vector KNN để trích xuất ra một tập nhỏ ứng viên tiềm năng nhất (khoảng 100 đến 200 ứng viên) trong thời gian vài mili-giây.

2. **Giai đoạn Xếp hạng chi tiết (Ranking Stage):** Sử dụng các mô hình học máy mạnh mẽ với hàng chục đặc trưng phức tạp để chấm điểm chính xác suất chuyển đổi cho từng sản phẩm trong tập ứng viên thô, sắp xếp và trả về Top-N sản phẩm tốt nhất.

## 2.3 Các phương pháp Candidate Generation (Recall Model)

Các mô hình triệu hồi phổ biến bao gồm:

- **Matrix Factorization (Phân rã ma trận):** Biến đổi ma trận tương tác User-Item thưa thớt thành tích vô hướng của hai ma trận embedding có số chiều ẩn thấp ( $d = 64$ ).
- **Deep Learning-based Recall:** Sử dụng các mạng thần kinh (như mạng hai tháp Two-Tower DSSM) để học vector nhúng biểu diễn ẩn cho User và Item.
- **Session-based Recall:** Khai thác các tương tác tức thì của người dùng trong phiên mua sắm hiện tại để tính toán vector bối cảnh, thực hiện triệu hồi các sản phẩm tương thích.

## 2.4 Các phương pháp Ranking (Ranking Model) và thuật toán LightGBM

Giai đoạn xếp hạng đòi hỏi mô hình phải có khả năng xử lý các đặc trưng dạng bảng phi tuyến phức tạp (giá, tần suất, bối cảnh thời gian) với tốc độ thực thi nhanh.

- **Các phương pháp xếp hạng:** Trực quan hóa điểm số CTR bằng Logistic Regression, Deep & Cross Networks (DCN), hoặc Gradient Boosting Decision Trees (GBDT).
- **Thuật toán LightGBM:** Khung thuật toán GBDT phát triển bởi Microsoft. LightGBM tối ưu hóa sự phân nhánh cây quyết định theo chiều dọc (Leaf-wise), kết hợp kỹ thuật lấy mẫu dựa trên gradient một bên (GOSS) và bỏ các đặc trưng loại trừ lẫn nhau (EFB), mang lại tốc độ huấn luyện siêu tốc và độ chính xác phân loại nhị phân cực tốt trên dữ liệu lớn.

## 2.5 Các chỉ số đánh giá hiệu suất (Evaluation Metrics)

- **Hit Rate@K (Tỷ lệ bao phủ):** Đo lường xem sản phẩm khách hàng thực sự tương tác có xuất hiện trong tập Top-K ứng viên triệu hồi hay không:

$$\text{Hit Rate@K} = \frac{1}{|U|} \sum_{u \in U} I(\text{Target}_u \in \text{Recall}_u^K) \quad (2.1)$$

- **NDCG@K (Normalized Discounted Cumulative Gain):** Đo lường độ chính xác của thứ tự xếp hạng, phạt nặng nếu sản phẩm mua sắm bị xếp ở cuối danh sách gợi ý:

$$\text{NDCG@K} = \frac{\text{DCG@K}}{\text{IDCG@K}} \quad \text{với} \quad \text{DCG@K} = \sum_{i=1}^K \frac{2^{rel_i} - 1}{\log_2(i + 1)} \quad (2.2)$$

- **MRR (Mean Reciprocal Rank):** Tính nghịch đảo vị trí của sản phẩm có tương tác đầu tiên, phản ánh nỗ lực cuộn trang của người dùng:

$$\text{MRR} = \frac{1}{|U|} \sum_{u \in U} \frac{1}{\text{rank}_u} \quad (2.3)$$

## Chương 3

# Quy trình Xử lý và Phân tích Dữ liệu

### 3.1 Phân tích Dữ liệu Khám phá (Exploratory Data Analysis - EDA)

Để có chiến lược xử lý dữ liệu và thiết kế mô hình chính xác, chúng em thực hiện khảo sát dữ liệu thô 2019-Oct.csv. Tập dữ liệu chứa thông tin chi tiết về các tương tác của khách hàng (User) lên sản phẩm (Item) theo thời gian thực bao gồm: event\_time, event\_type, product\_id, category\_id, category\_code, brand, price, user\_id, user\_session.

Kết quả phân tích khám phá chỉ ra hai đặc điểm vô cùng quan trọng:

1. **Độ thưa thớt ma trận (Sparsity) cực đoan:** Ma trận tương tác thưa tới **99.99%**. Tức là bình quân một khách hàng chỉ tương tác với 0.01% lượng sản phẩm có trên hệ thống, đòi hỏi phương pháp học embedding biểu diễn ẩn của Matrix Factorization.
2. **Phân phối lệch lớp hành vi:** Views chiếm **96.8%**, Carts chiếm **2.3%**, và Purchases chỉ chiếm **0.9%**. Do đó, mô hình rất dễ bị bão hòa bởi lượt xem thông thường nếu không áp dụng hàm Focal Loss hoặc Sample Weights để nhấn mạnh các hành vi chuyển đổi.
3. **Phân phối lệch phải của Price:** Giá cả sản phẩm tập trung ở phân khúc thấp nhưng có đuôi rất dài về phía phải. Ta cần áp dụng phép biến đổi Log-Transform:  $y = \log(x + 1)$  để kéo phân phối giá về dạng phân phối chuẩn, giúp mô hình học cây quyết định hội tụ nhanh và ổn định hơn.

### 3.2 Xử lý luồng dữ liệu (Data Pipeline) và cơ chế Sessionizer

Đường ống xử lý dữ liệu (Data Pipeline) được xây dựng theo kiến trúc tự động hóa tuần tự để chế biến dữ liệu thô thành dữ liệu huấn luyện và Feature Store phục vụ Serving:

- **Cơ chế Sessionizer (`sessionizer.py`):** Phân chia luồng tương tác của cùng một người dùng thành các phiên mua sắm độc lập. Một phiên mua sắm kết thúc khi không phát sinh thêm bất kỳ tương tác nào trong vòng **30 phút**.
- **Luồng thực thi tuần tự (`run_pipeline.py`):**
  1. Tiền xử lý dữ liệu: Fill NaN, ép kiểu dữ liệu và thực hiện Log-Transform Price.
  2. Chạy bộ Sessionizer phân chia và gán mã `custom_session_id`.
  3. Thực hiện trích xuất Feature Store snapshot tĩnh cho User và Item lưu thành Parquet.
  4. Trích xuất đặc trưng động Point-in-time lũy kế và gán nhãn implicit feedback lưu đĩa.

### 3.3 Kỹ thuật trích xuất đặc trưng (Feature Engineering)

Để ngăn chặn triệt để hiện tượng nghiêm trọng *Time-Travel Leakage* (Rò rỉ dữ liệu tương lai), chúng em áp dụng nguyên lý thiết kế đặc trưng lũy kế Point-in-time. Mọi đặc trưng chỉ được đếm và cộng dồn lũy kế dựa trên các sự kiện xảy ra TRƯỚC mốc *event\_time* hiện tại:

- **Đặc trưng User:** `user_total_interactions` (tổng tương tác lịch sử) và `user_total_sessions` (tổng số phiên hoạt động).
- **Đặc trưng Item:** `item_total_interactions` (sức hút của sản phẩm), `item_unique_users` (tập khách hàng quan tâm) và `item_avg_price` (mức giá trung bình lũy kế).
- **Đặc trưng Phiên động (Serving):** `user_session_interaction_count` (số tương tác của user trong phiên hiện tại tính đến thời điểm  $t$ ) và `item_session_popularity` (độ thịnh hành sản phẩm dựa trên số lượng phiên độc nhất).

### 3.4 Gán nhãn tương tác ngầm (Pseudo-Labeling)

Do dữ liệu thương mại điện tử không có nhãn đúng/sai rõ ràng, chúng em sử dụng kỹ thuật Pseudo-Labeling để quy đổi hành vi implicit feedback thành nhãn mục tiêu nhị phân nhằm phân tách rõ nét giữa ý định xem sản phẩm đơn thuần và hành vi chuyển đổi có giá trị cao. Trong mỗi phiên, chúng em gom nhóm theo người dùng và sản phẩm, giữ lại hành động có

mức độ cam kết lớn nhất làm nhãn huấn luyện mục tiêu:

$$\text{Label}(x) = \begin{cases} 1 & \text{nếu } x \in \{\text{Cart}, \text{Purchase}\} \\ 0 & \text{nếu } x \in \{\text{View}\} \text{ hoặc mẫu âm tính ngẫu nhiên (Negative Samples)} \end{cases} \quad (3.1)$$

Đồng thời gán `sample_weight` cho từng dòng dữ liệu: View nhận trọng số 0.1, Cart nhận trọng số 0.5, và Purchase nhận trọng số 1.0 để nhấn mạnh tầm quan trọng của hành vi chuyển đổi trong quá trình tối ưu hóa hàm tổn thất.



## Chương 4

# Thiết kế và Huấn luyện Mô hình

### 4.1 Kiến trúc tổng thể của hệ thống

Hệ thống được thiết kế theo kiến trúc gợi ý hai giai đoạn tích hợp triệu hồi Dual-Channel song song thời gian thực và xếp hạng lại bằng LightGBM. Mô hình triệu hồi PyTorch Matrix Factorization và FAISS Index FlatIP thực hiện sàng lọc thô cực nhanh để lọc ra 200 ứng viên từ 63,000+ sản phẩm. Sau đó, mô hình xếp hạng LightGBM chấm điểm CTR chi tiết, sắp xếp và trả về Top 20 sản phẩm tối ưu.

### 4.2 Xây dựng và tối ưu mô hình Recall

Mô hình Recall được xây dựng bằng mạng học sâu Matrix Factorization trên PyTorch:

- **Cấu trúc mô hình:** Lớp nhúng (`nn.Embedding`) ánh xạ các ID thô của User và Item thành các vector ẩn 64 chiều. Trọng số embeddings được khởi tạo bằng phương pháp **Xavier Uniform** giúp đảm bảo tính ổn định và tốc độ hội tụ nhanh.
- **Negative Sampling 1:4:** Với mỗi tương tác dương lịch sử (label 1), chúng em lấy mẫu ngẫu nhiên 4 sản phẩm người dùng chưa từng tương tác làm mẫu âm (label 0) để cân bằng dữ liệu huấn luyện.
- **Hàm tổn thất Focal Loss:** Để giải quyết mất cân bằng lớp cực đoan (View chiếm 96.8% lượt tương tác), chúng em phát triển lớp `FocalLoss` tùy chỉnh với  $\alpha = 0.25$  và  $\gamma = 2.0$ :

$$FL(p_t) = -\alpha_t(1 - p_t)^\gamma \log(p_t) \quad (4.1)$$

Focal Loss chủ động dập giảm lỗi từ các mẫu xem dễ dự đoán ( $p_t \approx 0.9$ ), ép mô hình tập trung dồn gradient học các tương tác hiếm và quan trọng như Cart (Thêm giỏ) và

Purchase (Mua hàng).

- **Chỉ mục FAISS Index FlatIP:** Tải 63,322 vector những sản phẩm đã huấn luyện lên RAM để dựng chỉ mục tích vô hướng FAISS KNN FlatIP siêu tốc, cho phép quét 100 sản phẩm tương quan nhất dưới 0.02 mili-giây.

### 4.3 Xây dựng và huấn luyện mô hình Ranking

Mô hình xếp hạng LightGBM GBDT được thiết lập để chấm điểm chi tiết xác suất CTR:

- **Phân chia tập dữ liệu Time-based Splitting:** Sắp xếp các tương tác theo thời gian và phân chia theo tỷ lệ **80% Train / 20% Test** dựa trên mốc `event_time`, đảm bảo mô phỏng chính xác quá trình phục vụ trực tuyến.
- **Tham số cấu hình LightGBM:** Thiết lập siêu tham số `is_unbalance=True`, `num_leaves=31`, `learning_rate=0.05` và `num_boost_round=100`.
- **Trọng số mẫu (Sample Weights):** Truyền trực tiếp trọng số tương tác implicit ngầm (`view=0.1`, `cart=0.5`, `purchase=1.0`) vào `lgb.Dataset` để nhấn mạnh hành vi mua hàng.

### 4.4 Giải pháp Train-on-Recall và Kết quả thử nghiệm

Trong mô hình cơ sở (Baseline), LightGBM được huấn luyện trên toàn bộ dữ liệu tĩnh, dẫn đến lỗi lệch phân phối dữ liệu huấn luyện xếp hạng (Train-Test Distribution Mismatch).

Để giải quyết triệt để, chúng em phát triển giải pháp cải tiến cực hạn **Train-on-Recall** (`train_on_recall.py`):

1. Duyệt qua 20,000 phiên, gọi mô hình PyTorch Recall và FAISS triệu hồi ra 200 ứng viên (giống hệt 100% môi trường Serving thực tế).
2. Gán nhãn ứng viên: Trở thành nhãn dương (1) nếu trùng khớp với tương tác thực tế của phiên, và nhãn âm (0) (với `sample weight 0.1`) cho các ứng viên không tương tác.
3. Huấn luyện LightGBM trên chính không gian ứng viên triệu hồi này kết hợp với hai cờ chỉ thị nguồn gốc triệu hồi nhị phân `recalled_by_long_term` và `recalled_by_session`.

Kết quả thử nghiệm ngoại tuyến (Offline Evaluation) cho thấy sự vượt trội kinh ngạc của mô hình cải tiến:

Bảng 4.1: Bảng so sánh hiệu năng xếp hạng giữa Baseline và mô hình cải tiến Train-on-Recall

| Độ đo đánh giá (Metrics) | Mô hình Baseline | Mô hình Train-on-Recall | Mức cải thiện |
|--------------------------|------------------|-------------------------|---------------|
| <b>Validation AUC</b>    | 0.6375           | <b>0.9601</b>           | <b>+50.6%</b> |
| <b>NDCG@10</b>           | 0.0495           | <b>0.0653</b>           | <b>+31.9%</b> |
| <b>MRR</b>               | 0.0458           | <b>0.0572</b>           | <b>+24.9%</b> |

Chỉ số AUC tăng vọt đột biến lên **0.9601** chứng tỏ cây quyết định LightGBM đã học được cách phân biệt cực kỳ chính xác giữa tương tác thực tế và các sản phẩm bị người dùng bỏ qua (True Negatives) trong không gian ứng viên thô của FAISS. Sự bổ sung cờ triệu hồi Kênh phiên giúp NDCG@10 tăng vọt **31.9%** và MRR tăng **24.9%**.

## Chương 5

# Triển khai Hệ thống Phục vụ (Serving Pipeline)

### 5.1 Thiết kế luồng phục vụ hệ thống (System Flow)

Serving Pipeline là cầu nối trực tiếp tiếp nhận các yêu cầu truy vấn của người dùng thời gian thực thông qua API và trả về kết quả khuyến nghị cá nhân hóa. Chúng em xây dựng máy chủ FastAPI và Uvicorn hiệu năng cao:

- **In-Memory Feature Tables:** Tải toàn bộ thuộc tính tĩnh Parquet (`user_features.parquet`, `item_features.parquet`) và FAISS Index lên bộ nhớ RAM ngay khi API khởi động, triệt tiêu hoàn toàn nghẽn Disk I/O.
- **Luồng vận hành Serving:**
  1. Tiếp nhận yêu cầu GET `/recommend/{user_id}?session_items=A,B,C` chứa ID người dùng và lịch sử các sản phẩm vừa click tương tác trong phiên.
  2. Phân phối song song hai kênh triệu hồi FAISS (Sở thích lâu dài & Phiên tức thì).
  3. Gộp ứng viên và loại trùng lặp (loại bỏ các mặt hàng trùng nhau).
  4. Trích xuất thuộc tính tĩnh từ RAM Feature Store, ghép nối với đặc trưng phiên tính động tại thời điểm API chạy.
  5. Đưa dữ liệu bảng vào LightGBM Ranker để chấm điểm xác suất chuyển đổi CTR kháng lỗi.
  6. Sắp xếp điểm số giảm dần và trả về Top 20 sản phẩm khuyến nghị tốt nhất định dạng JSON.

## 5.2 Truy xuất vector lân cận gần nhất siêu tốc bằng FAISS Index

Để đảm bảo khâu triệu hồi thô diễn ra trong chưa đầy 3ms, chúng em sử dụng thư viện **FAISS (Facebook AI Similarity Search)** viết bằng C++:

- Dụng chỉ mục tìm kiếm vector xấp xỉ Flat Inner Product (`faiss.IndexFlatIP`) dựa trên tích vô hướng Dot Product của embeddings ẩn 64 chiều.
- Tìm kiếm song song: Truy vấn cùng lúc 100 sản phẩm lân cận cho kênh lâu dài và 100 sản phẩm lân cận cho kênh phiên tức thì (thông qua Average Session Embedding).
- FAISS cho phép thực hiện tìm kiếm vector trên RAM chỉ tốn **0.013 giây** cho cơ sở dữ liệu **24,428 sản phẩm** lúc khởi động, đảm bảo thời gian quét vector phục vụ API chỉ mất dưới **2.24ms**.

## 5.3 Tích hợp Pipeline và cơ chế Ranker thời gian thực kháng lỗi

Khâu xếp hạng lại trực tuyến LightGBM GBDT được thiết lập cơ chế **kháng lỗi suy luận động (Fault-Tolerant Serving)** vô cùng mạnh mẽ tại `src/serving/ranker.py`:

- **Dynamic Feature Alignment (Tự động so khớp đặc trưng):** Trong môi trường thực tế, nếu mô hình LightGBM được huấn luyện với các cột đặc trưng bổ sung nhưng Feature Store Serving bị thiếu cột, LightGBM sẽ crash. Hệ thống tự động gọi `self.model.feature_names_in_` lúc khởi động để trích xuất danh sách cột mô hình mong đợi. Sau đó, nó tự động đối chiếu, bù đắp các cột thiếu bằng giá trị mặc định (<UNKNOWN> cho phân loại, 0.0 cho số học) và sắp xếp lại các cột theo đúng thứ tự mô hình yêu cầu trước khi chấm điểm. Điều này đảm bảo API không bao giờ bị gián đoạn hay crash.
- **Cold-Start Fallback:** Nếu gặp một `user_id` mới hoàn toàn (chưa có trong Feature Store), hệ thống tự động chuyển hướng gợi ý danh sách sản phẩm thịnh hành nhất (`_popular_fallback_recommendations`) dựa trên tương tác lịch sử thực tế mà không gây lỗi runtime.

## 5.4 Kiểm thử hệ thống (Smoke Testing)

Để xác thực tính đúng đắn và hiệu năng thực tế của luồng phục vụ, chúng em phát triển kịch bản kiểm thử tích hợp tự động **Smoke Testing** (`tests/smoke_test.py`):

1. Khởi chạy tự động Pipeline, nạp encoders, nạp PyTorch MF model và LightGBM ranker.
2. Kiểm tra luồng khởi động: Pipeline sẵn sàng phục vụ chỉ sau **0.85 giây**.
3. Mô phỏng truy vấn Serving cho User ID 244951053. Kết quả suy luận thành công mượt mà, trả về đúng định dạng cấu trúc JSON chứa Top 5 sản phẩm đề xuất tốt nhất.
4. Tổng độ trễ Serving được đo lường thực tế đạt mức rất thấp: **16.89 ms** (nằm sâu dưới giới hạn tiêu chuẩn công nghiệp 50ms).

Bảng 5.1: Bảng đo lường độ trễ chi tiết các bước trong API Serving trực tuyến

| Bước phục vụ (Steps)                      | Độ trễ đo được (ms) | Tỷ trọng (%) |
|-------------------------------------------|---------------------|--------------|
| 1. Tra cứu Feature Store RAM              | 2.8 ms              | 12.4%        |
| 2. Triệu hồi vector FAISS (Dual-Channel)  | 4.5 ms              | 20.0%        |
| 3. Chấm điểm xếp hạng LightGBM GBDT       | 12.2 ms             | 54.2%        |
| 4. Gom nhóm và trả về JSON API            | 3.0 ms              | 13.4%        |
| <b>Tổng độ trễ toàn chu trình (Total)</b> | <b>22.5 ms</b>      | <b>100%</b>  |

## Chương 6

# Kết luận và Hướng phát triển

### 6.1 Tổng kết các kết quả đạt được

Dự án nghiên cứu xây dựng **Hệ thống Gợi ý Hai Giai Đoạn tiêu chuẩn công nghiệp** đã hoàn thành trọn vẹn và đạt được các kết quả xuất sắc về mặt học thuật lẫn thực chiến công nghiệp: 1. **Kiến trúc Two-Stage vững chắc:** Triển khai thành công phễu lọc thô triệu hồi PyTorch + FAISS song song hai kênh (gu lâu dài & bối cảnh phiên tức thì) cùng khâu xếp hạng chi tiết LightGBM GBDT, đảm bảo phản hồi trực tuyến siêu tốc **22.5 ms** (vượt xa yêu cầu công nghiệp < 50ms). 2. **Cơ chế kháng rò rỉ Point-in-time:** Loại bỏ hoàn toàn lỗi Data Leakage nghiêm trọng bằng đường ống trích xuất đặc trưng lũy kế progressive Point-in-time. 3. **Giải quyết lệch phân phối Train-on-Recall:** Xóa bỏ triệt để lỗi Train-Test Distribution Mismatch thông qua kịch bản huấn luyện xếp hạng trên chính không gian ứng viên triệu hồi thực tế, giúp tăng vọt Validation AUC lên **0.9601** (+50.6%) và NDCG@10 tăng **31.9%**. 4. **Serving kháng lỗi tối ưu:** Xây dựng API FastAPI in-memory hiệu năng cao tích hợp cơ chế tự động bù đặc trưng thiếu và Cold-Start Fallback, đảm bảo API vận hành bền bỉ 24/7.

### 6.2 Phân tích các vấn đề dữ liệu còn tồn đọng (Data Issues)

Mặc dù hệ thống đạt được hiệu năng và độ chính xác vô cùng ấn tượng, quá trình audit codebase đã phát hiện một số vấn đề dữ liệu còn tồn đọng cần được giải quyết trong giai đoạn tiếp theo: \* **Tính chất mất cân bằng lớp cực đoan:** Tỷ lệ chuyển đổi dương (label 1) chỉ chiếm vón vẹn 0.09% trong tập kiểm thử xếp hạng (299 tương tác chuyển đổi trên 320,000+ dòng dữ liệu). Hầu hết các phiên kiểm thử không chứa hành vi mua hàng thực tế làm NDCG trung bình toàn hệ thống bị kéo thấp xuống. \* **Đánh giá Recall chưa cách ly hoàn toàn:** Giai đoạn triệu hồi thô hiện tại đang đo lường Hit Rate@50 trực tiếp trên cùng tập dữ liệu huấn luyện do kích thước dữ liệu snapshot hạn chế. Điều này tạo ra một khoảng

trống đánh giá (Evaluation Gap), có nguy cơ quá khớp (overfitting) mà trệnh nhúng của người dùng và sản phẩm.

## 6.3 Kế hoạch cải thiện dữ liệu và hệ thống (Data Improvement Plan)

Để tiếp tục nâng cao hiệu năng hệ thống lên tầm cao mới, chúng em đề xuất kế hoạch cải thiện dữ liệu và nâng cấp hệ thống chi tiết bao gồm:

### 6.3.1 Kế hoạch cải thiện Dữ liệu (Data Improvement Plan)

- **Tăng cường tập mẫu dương (Positive Sampling Augmentation):** Thu thập thêm dữ liệu tương tác trong các khung giờ cao điểm mua sắm, áp dụng kỹ thuật sinh mẫu nhân tạo hoặc lọc bớt các phiên trống không phát sinh chuyển đổi để nâng tỷ lệ nhãn dương của tập xếp hạng từ 0.09% lên khoảng 1% – 2%, giúp cây quyết định LightGBM học được nhiều phân ngưỡng phân nhánh tối ưu hơn.
- **Cách ly dữ liệu đánh giá Recall (Recall Evaluation Isolation):** Thực hiện phân chia tập dữ liệu huấn luyện và kiểm thử theo thời gian (Time-based Splitting) tách biệt hoàn toàn cho cả giai đoạn Recall ban đầu, đo lường Hit Rate trên tập test độc lập để kiểm chứng khách quan năng lực tổng quát hóa của embeddings.

### 6.3.2 Kế hoạch nâng cấp Hệ thống (System Improvement Plan)

- **Huấn luyện Recall bằng BPR Loss (Bayesian Personalized Ranking):** Chuyển từ tối ưu phân loại độc lập từng mẫu (Focal Loss) sang tối ưu hóa thứ tự ưu tiên cặp tương quan (Pairwise Ranking) để đảm bảo sản phẩm đã tương tác luôn có điểm dot product cao hơn sản phẩm chưa tương tác, nâng cao chất lượng lọc thô của bộ triệu hồi.
- **Tối ưu hóa chỉ mục FAISS HNSW:** Khi quy mô sản phẩm vượt ngưỡng 100,000+, chuyển đổi sang chỉ mục xấp xỉ phân tầng HNSW để duy trì tốc độ tìm kiếm vector KNN dưới 5ms.
- **Tích hợp Feast Feature Store:** Triển khai Feast Feature Store chuyên dụng chuẩn công nghiệp để quản lý đồng bộ Online Feature Store (cho API phục vụ) và Offline Feature Store (cho đường ống huấn luyện), tự động hóa luồng cập nhật đặc trưng động.